LANGCHAIN 마이그레이션 도우미 · 4번(검증·평가) 담당 안내서

검증·평가 파트 상세

최신 버전 · 2026-10-12

4번 담당이 맡은 **요리 대회 심사위원** 임무를 자세히 설명합니다. 도구, 명령어, 결과 읽는 법, 발표 질문까지 담았습니다.

먼저 읽을 문서: 01 프로젝트 전체 쉽게 보기

이것 세 줄만 기억해도 됩니다

1. AI는 **옛날 주방**만 기억해서, 도구가 이사 간 줄 모르고 옛날 레시피를 줍니다.
2. 우리 프로그램은 **새 주방 안내서를 보고** 레시피를 고친 뒤, **서랍을 직접 열어 보며** 도구가 있는지 확인합니다.
3. 나(4번)는 **요리 대회 심사위원**입니다. 같은 문제 30개로 "우리가 더 잘 고친다"를 숫자로 증명합니다.

1장

내 임무: 요리 대회 심사위원

요리사 두 명에게 **똑같은 옛날 레시피 30개**를 주고 "새 주방에 맞게 고쳐 오세요"라고 합니다. 한 명은 **일반 AI**, 한 명은 **우리 프로그램**입니다. 나는 고쳐 온 레시피대로 서랍을 실제로 열어 보고, 도구가 다 있으면 합격을 줍니다.

내 도구 상자 (모두 만들어져 있습니다)

파일	주방 비유	내가 할 일
`EVAL_RULES.md`	시합 규칙표	팀과 빈칸 2개 채우기 (비교 모델, verifier 담당)
`testset.json`	문제지 30장	읽어 보고 왜 깨지는지 몇 개 설명할 수 있게 하기
`check_testset.py`	문제지 검사기	한 번 실행해서 "통과 30/30" 눈으로 보기
`verify.py`	채점 로봇	1번 팀원에게 "우리 시스템도 이걸로 채점해 주세요" 전달
`run_baseline.py`	상대 선수 시험 기계	일반 AI 시험 실행
`make_charts.py`	점수판 그리기 기계	결과가 나오면 실행해서 그래프 받기

왜 AI에게 "이거 맞아?"라고 묻지 않나요?

심사위원도 옛날 주방만 기억하면, 틀린 레시피에 합격을 줄 수 있습니다. 그래서 우리는 **말이 아니라 서랍을 직접 열어 보는 것**으로 채점합니다. 이게 일반 AI와 우리의 가장 큰 차이입니다.

공정한 시합 규칙 (달리기와 같습니다)

규칙	달리기로 말하면
두 요리사는 같은 AI 두뇌(모델)를 쓴다	같은 운동화를 신는다
같은 문제 30개를 같은 순서로 푼다	같은 코스를 뛴다
같은 채점 로봇(`verify.py`)으로 채점한다	같은 시계로 잰다

꼼수는 실패로 친다 (어려운 부분을 지워서 에러를 없애기 등)	코스를 가로지르면 실격

꼭 알아야 하는 것

왜 서랍을 직접 열어 보는지, 시합이 왜 공정한지, 결과 숫자가 무슨 뜻인지, 그리고 한계(아래).

몰라도 괜찮은 것

파이썬 코드 한 줄 한 줄의 뜻. 실행하는 방법만 알면 됩니다. 막히면 Claude에게 물어보면 됩니다.

한계 (발표에서 먼저 말하면 신뢰가 올라갑니다)

서랍에서 도구를 **꺼낼 수 있다**고 해서, 그 도구를 **올바르게 쓴다**는 보장은 없습니다. 예를 들어 국자는 꺼냈는데 국자로 칼질을 하면 요리는 망합니다. 우리 채점은 "꺼낼 수 있나"까지만 확인합니다.

2장

따라 하기: 명령어와 화면에 나오는 것

아래 명령어를 순서대로 PowerShell이나 터미널에 붙여 넣으면 됩니다. 각 단계마다 **화면에 이렇게 나오면 성공**을 함께 적었습니다.

0단계. 평가 폴더로 이동

```
cd "C:\Users\playdata2\Desktop\skn35 report\langchain_migration\eval"
```

1단계. 새 주방 만들기 (처음 한 번만)

최신 LangChain만 설치된 깨끗한 연습용 주방(가상환경)을 따로 만듭니다. 2~3분 걸립니다.

```
uv venv verify-env --python 3.12
uv pip install --python verify-env/Scripts/python.exe langchain langchain-core
langchain-openai langchain-community langchain-classic langgraph
verify-env/Scripts/python.exe -m pip freeze > requirements-verify.txt
```

마지막 줄은 "오늘 주방에 어떤 도구 몇 번 버전이 있었는지" 기록하는 것입니다. 실험이 끝날 때까지 이 주방은 바꾸지 않습니다.

2단계. 문제지 검사 (눈으로 확인하기)

```
python check_testset.py testset.json --py verify-env/Scripts/python.exe
```

이렇게 나오면 성공입니다. 30문제 모두 "옛날 서랍을 열면 에러가 난다"는 뜻입니다.

```
[PASS] easy_01 (easy)
        깨지는 줄: from langchain.chat_models import ChatOpenAI
          → ImportError: cannot import name 'ChatOpenAI' ...
...
== 요약 ==
통과: 30 / 30
난이도별 개수: easy 10, medium 10, hard 10  (목표: 각 10개)
```

3단계. 상대 선수(일반 AI) 시험 보기

먼저 OpenAI 열쇠(API 키)를 넣습니다. `sk-...` 부분에 **내 키**를 넣으세요. 키는 절대 GitHub에 올리면 안 됩니다.

```
pip install openai matplotlib
$env:OPENAI_API_KEY="sk-...내 키..."
```

처음엔 3문제만 시험해서 잘 되는지 봅니다. `[모델명]`은 팀이 정한 모델로 바꿉니다.

```
python run_baseline.py --py verify-env/Scripts/python.exe --model [모델명] --limit 3
```

```
[1/3] easy_01 ... 성공
[2/3] easy_02 ... 실패 (import: ModuleNotFoundError: No module named
'langchain.llms')
...
=== baseline(A) 결과 ===
전체 성공률: ?/3 = ?%
```

문제없으면 `--limit 3`을 빼고 30문제 전체를 실행합니다. 중간에 멈춰도 다시 실행하면 이어서 합니다.

4단계. 우리 프로그램 시험 보기

이건 1번 팀원이 만든 프로그램으로 실행합니다. 나는 **"채점은 verify.py로, 성적표는 EVAL_RULES 7장 모양으로 저장해 주세요"**라고 부탁만 하면 됩니다.

5단계. 점수판 그리기

```
python make_charts.py --a results/A_[모델명]_[날짜].json --b results/B_[모델명]_[날짜].json
```

`charts` 폴더에 그래프 3장과 `summary.md`(숫자 표)가 생깁니다. 이걸 발표 자료와 README에 넣으면 끝입니다.

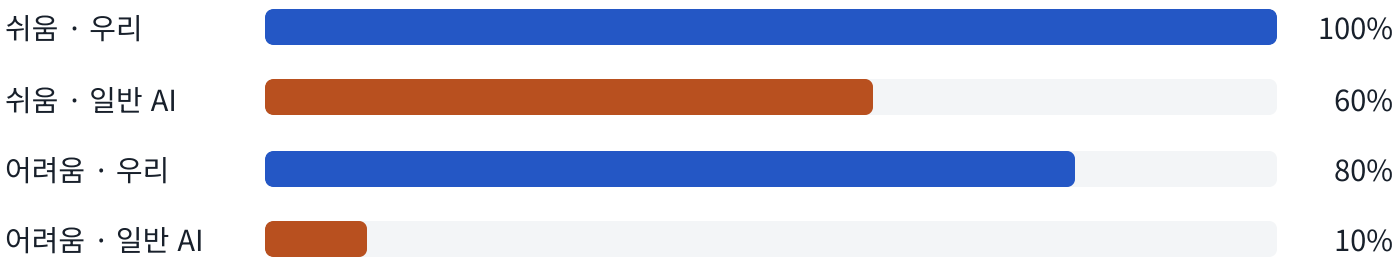
> **막히면 Claude에게 이렇게 물어보세요**
>
> "langchain_migration/eval 폴더에서 *[실행한 명령어]*를 실행했더니 이런 에러가 났어. *[에러 메시지 전체 붙여넣기]* 중학생도 이해하게 원인이랑 해결 방법 알려줘."

3장

결과 읽는 법

아래 숫자는 **읽는 연습용 예시**입니다. 진짜 결과가 아닙니다.

예시: 난이도별 성공률



읽는 법: 쉬운 문제는 차이가 40%p, 어려운 문제는 70%p입니다. → "어려운 레시피일수록 옛날 기억만으로는 못 고친다"고 말할 수 있습니다.

꼭 알아야 할 숫자 3개

숫자	뜻	예시로 계산하면
1회차 성공률	우리 프로그램이 처음 한 번에 맞힌 비율	30개 중 6개 → 20%
최종 성공률	다시 찾고 고치기를 최대 3번까지 해서 맞힌 비율	30개 중 27개 → 90%
검증 루프 효과	최종 − 1회차. "서랍을 열어 보고 다시 고치기"가 만든 차이	90 − 20 = **+70%p**

%p(퍼센트 포인트)는 퍼센트끼리 뺀 값입니다. 20%에서 90%가 되면 "70%p 올랐다"고 말합니다.

성공에도 품질이 있습니다

종류	주방 비유
깔끔한 성공	새 서랍에서 새 도구를 꺼냄
성공 (community 경고)	"이번 달까지만 영업" 가게에서 도구를 사 옴. 지금은 되지만 곧 못 씀

성공 (classic 의존)	창고에 치워 둔 낡은 도구를 꺼내 씀. 되긴 하지만 요즘 방식은 아님
실패	서랍을 열었는데 도구가 없음

"community 가게가 곧 문 닫는다"는 사실은 **우리가 실험하다가 직접 발견한 것**입니다(2026-10-12). 발표에서 좋은 이야깃거리가 됩니다.

4장

발표 질문 대비: 쉬운 말로 답하기

예상 질문	이렇게 답하세요
import만 확인하면 충분한가요?	"아닙니다. 도구를 꺼낼 수 있는지만 확인하고, 올바르게 쓰는지는 확인하지 못합니다. 가장 흔한 고장이 import 에러라서 여기부터 확실하게 잡았고, 실제 실행 테스트는 다음 단계입니다."
왜 AI에게 검증을 맡기지 않았나요?	"검증하는 AI도 같은 옛날 지식을 가질 수 있어서, 틀린 코드를 맞다고 할 수 있습니다. 그래서 실제로 실행해 보는 방법을 택했습니다."
30개로 충분한가요?	"통계적으로 일반화하려는 목적보다, 난이도별로 차이가 어떻게 달라지는지 경향을 보려는 목적입니다. 쉬움·중간·어려움을 10개씩 균형 있게 넣었습니다."
문제를 AI로 만들었으면 불공평하지 않나요?	"공식 마이그레이션 가이드에 나온 변경 사항을 기준으로 만들었고, 30개 모두 최신 환경에서 실제로 깨지는 것을 확인했습니다. 두 선수 모두 같은 문제를 풀었습니다."
일반 AI에 문서를 주면 되지 않나요?	"연구에 따르면 문서를 줘도 실행 성공률이 66% 정도에 머뭅니다. 그래서 문서에 더해 실행 확인과 재시도가 필요합니다."
실무에서 어떤 가치가 있나요?	"버전이 바뀔 때마다 개발자가 에러를 하나씩 찾아 고치는 시간을 줄여 줍니다. 근거 링크와 검증 결과가 함께 나와서 믿고 쓸 수 있고, 안내서와 주방만 바꾸면 다른 라이브러리에도 쓸 수 있습니다."

5장

확인 퀴즈

답을 누르면 바로 맞았는지 알려 줍니다. **0 / 5 맞힘**

Q1. 왜 AI에게 "이 코드 맞아?"라고 물어서 채점하지 않나요?

돈이 많이 들어서

채점하는 AI도 옛날 주방만 기억할 수 있어서

AI는 한국어를 못 해서

> 해설: 심사위원도 옛날 기억만 있으면 틀린 레시피에 합격을 줄 수 있습니다. 그래서 직접 실행해서 확인합니다.

Q2. import가 성공하면 코드가 완벽하다고 말할 수 있나요?

네, 완벽하다

아니요. 도구를 꺼낼 수는 있지만 올바르게 쓰는지는 모른다

> 해설: 국자를 꺼냈다고 국자를 제대로 쓰는 건 아닙니다. 이게 우리 채점의 한계이고, 발표에서 먼저 말해야 합니다.

Q3. baseline은 무슨 뜻인가요?

우리 시스템의 최종 점수

비교 기준. 여기서는 일반 AI의 성적

시험 문제 30개

> 해설: "우리가 더 잘한다"를 말하려면 비교할 기준이 필요합니다. 그게 baseline입니다.

Q4. 공정한 시합 규칙이 아닌 것은?

- 두 선수가 같은 AI 모델을 쓴다
- 같은 문제 30개를 푼다
- 우리 시스템에만 더 쉬운 문제를 준다
- 같은 채점 로봇으로 채점한다

> 해설: 같은 코스를 뛰어야 공정한 달리기입니다. 문제를 다르게 주면 비교가 의미 없어집니다.

Q5. 1회차 성공률 20%, 최종 성공률 90%일 때 검증 루프 효과는?

- +20%p
- +70%p
- +90%p

> 해설: 최종 − 1회차 = 90 − 20 = 70. 퍼센트끼리 뺀 값이라 %p를 붙입니다.

6장

이번 주 체크리스트

체크한 내용은 이 브라우저에만 저장됩니다. **0 / 11 완료**

- [ ] 이 안내서를 끝까지 읽고 퀴즈 8개 중 6개 이상 맞히기
  틀린 문제는 해설을 다시 읽기
- [ ] 팀과 비교 모델 하나 정하기
  EVAL_RULES.md 9장 빈칸 채우기
- [ ] 1번 팀원에게 verifier 구현과 결과 형식 부탁하기
  "채점은 verify.py로, 성적표는 EVAL_RULES 7장 모양으로"
- [ ] 새 주방(verify-env) 만들고 requirements-verify.txt 저장하기
  2장 1단계
- [ ] check_testset.py 실행해서 통과 30/30 눈으로 보기
  2장 2단계
- [ ] testset.json에서 쉬움·중간·어려움 하나씩 골라 왜 깨지는지 말로 설명해 보기
  발표 질문 대비
- [ ] 일반 AI 시험(run_baseline.py) 3문제 → 30문제 실행하기
  2장 3단계
- [ ] 우리 시스템 결과 받기
  1번 팀원
- [ ] make_charts.py로 그래프 3장 만들기
  2장 5단계
- [ ] 발표 자료 [__] 칸에 숫자와 그래프 넣기
  8~10번 슬라이드

- [ ] 4장 예상 질문 6개를 소리 내어 답해 보기
  리허설

최신 버전 · 2026-10-12 · 같은 내용의 MD와 PDF는 저장소 `docs/guide` 폴더에 있습니다. 이전 버전: `docs/역순설계_쉽게_이해하기.md`